

Final Phase: Array ADT Capstone Project — Full Structured Notes

1. Big Idea of the Final Phase

The final phase is the **Array ADT project phase**.

This phase is not mainly about learning one new operation. Instead, it is about combining everything you already learned into one complete program.

Simple idea:

```
Final Array ADT Project =  
  Array structure  
  + all Array ADT functions  
  + menu system  
  + input validation  
  + dry runs  
  + complexity understanding
```

By the end of this phase, you should be able to build a small **menu-driven Array ADT program** that lets the user choose different operations such as display, append, insert, delete, search, reverse, merge, union, intersection, and difference.

2. Current Curriculum Progress

Before the final project, you already covered all main concept phases.

Phase	Main Topic	What You Learned
Phase 1	Array ADT foundation	<code>A</code> , <code>size</code> , <code>length</code> , static arrays, dynamic arrays
Phase 2	Core update operations	display, append, insert, delete
Phase 3	Searching	linear search, transposition, move-to-front
Phase 4	Binary search	iterative binary search, recursive binary search, complexity analysis

Phase	Main Topic	What You Learned
Phase 5	Basic operations	get, set, max, min, sum, recursive sum, average
Phase 6	Rearrangement	reverse, shift, rotate
Phase 7	Sorted array operations	sorted insertion, checking sorted order, rearranging negative and positive values
Phase 8	Binary and set operations	merge, union, intersection, difference, membership

So the final phase is the **integration phase**.

You are now expected to connect these operations into one working project.

3. What the Final Phase Is Really Testing

The final phase checks whether you understand how the whole Array ADT works together.

It tests these skills:

1. Can you define the correct `struct Array`?
2. Can you use `size` and `length` correctly?
3. Can you write functions that modify the real array?
4. Can you write functions that only read the array?
5. Can you use a menu to call different functions?
6. Can you validate indexes before using them?
7. Can you avoid accessing unused array slots?
8. Can you use binary search only on sorted arrays?
9. Can you create result arrays for merge and set operations?
10. Can you explain time and space complexity?
11. Can you dry-run important operations by hand?

Simple memory:

The final phase proves that you can build and control the full Array ADT system.

4. Main Goal of the Capstone Project

The final project should behave like a small Array ADT toolbox.

The user sees a menu like this:

```
1. Display
2. Append
3. Insert
4. Delete
5. Linear Search
6. Binary Search
7. Get
8. Set
9. Max
10. Min
11. Sum
12. Average
13. Reverse
14. Merge
15. Union
16. Intersection
17. Difference
0. Exit
```

Enter your choice:

Then the user chooses an operation.

The program asks for the needed input.

The function runs.

The result is shown.

Then the menu appears again.

Simple flow:

```
Show menu
Get user choice
Call the correct function
Show result
Repeat until user exits
```

5. What Is a Menu-Driven Program?

A **menu-driven program** is a program where the user chooses actions from a list.

Example:

```
1. Display
2. Append
3. Insert
0. Exit
Enter choice:
```

If the user enters , the program displays the array.

If the user enters , the program asks for a value and appends it.

If the user enters , the program asks for an index and value, then inserts the value.

If the user enters , the program stops.

The menu is usually controlled with:

```
do {
    // show menu
    // read choice
    // switch(choice)
} while (choice != 0);
```

6. Basic Menu Structure

A simple menu pattern looks like this:

```
do {
    printf("\n--- Array ADT Menu ---\n");
    printf("1. Display\n");
    printf("2. Append\n");
    printf("3. Insert\n");
    printf("4. Delete\n");
    printf("0. Exit\n");
    printf("Enter choice: ");
    scanf("%d", &choice);
```

```

switch (choice) {
    case 1:
        display(arr);
        break;

    case 2:
        printf("Enter value: ");
        scanf("%d", &x);
        append(&arr, x);
        break;

    case 3:
        printf("Enter index and value: ");
        scanf("%d %d", &index, &x);
        insert(&arr, index, x);
        break;

    case 4:
        printf("Enter index: ");
        scanf("%d", &index);
        printf("Deleted value: %d\n", Delete(&arr, index));
        break;

    case 0:
        printf("Exiting...\n");
        break;

    default:
        printf("Invalid choice\n");
}
} while (choice != 0);

```

Important point:

The menu does not do the real array work itself.

The menu only controls which function gets called.

7. Array ADT Structure Used in the Final Project

The project should use a structure like this:

```
#define MAX 50

struct Array {
    int A[MAX];
    int size;
    int length;
};
```

Meaning:

Part	Meaning
A	The actual array storage
size	Maximum capacity of the array
length	Number of valid elements currently stored

Example:

```
struct Array arr = {{2, 4, 6, 8}, 50, 4};
```

This means:

```
A = [2, 4, 6, 8, _, _, _, ...]
size = 50
length = 4
```

Only indexes 0 to 3 are valid.

Index 4 is the first free slot.

8. Most Important Rule: size vs length

This is one of the most important rules in the whole project.

```
size = total capacity
length = number of valid elements
```

Example:

```
struct Array arr = {{2, 4, 6, 8}, 50, 4};
```

Here:

```
size = 50  
length = 4
```

That means the array has space for 50 values, but only 4 values are currently valid.

Correct display loop:

```
for (int i = 0; i < arr.length; i++) {  
    printf("%d ", arr.A[i]);  
}
```

Wrong display loop:

```
for (int i = 0; i < arr.size; i++) {  
    printf("%d ", arr.A[i]);  
}
```

Why is the second one wrong?

Because it prints unused slots also.

Simple memory:

```
Use length when processing real data.  
Use size only when checking capacity.
```

9. Function Passing Rule

Some functions only read the array.

Some functions modify the array.

This decides whether we pass the structure by value or by address.

10. Pass by Value

Use **pass by value** when the function only reads the array.

Example:

```
void display(struct Array arr)
```

This function only prints values.

It does not change:

- array values
- size
- length

So pass by value is acceptable.

Examples of read-only functions:

```
display(arr);  
get(arr, index);  
max(arr);  
min(arr);  
sum(arr);  
average(arr);  
linearSearch(arr, key);  
BinarySearch(arr, key);
```

Inside pass-by-value functions, use dot notation:

```
arr.length  
arr.A[i]
```

11. Pass by Address

Use **pass by address** when the function modifies the real array.

Example:


```
void append(struct Array *arr, int x)
```

Append changes:

- the array values
- `length`

So it must receive the address of the structure.

Examples of modifying functions:

```
append(&arr, x);
insert(&arr, index, x);
Delete(&arr, index);
set(&arr, index, x);
Reverse2(&arr);
leftShift(&arr);
rightRotate(&arr);
insertSort(&arr, x);
rearrange(&arr);
```

Inside pointer-based functions, use arrow notation:

```
arr->length
arr->A[i]
```

Simple memory:

```
Read only? Pass by value.
Modify array? Pass by address.
```

12. Three Categories of Operations

Every operation in the project belongs to one of three main categories.

Category	Examples	Main Rule
Read-only operations	display, get, max, min, sum, search	Pass by value is okay

Category	Examples	Main Rule
Modify same array	append, insert, delete, set, reverse, shift, rotate	Pass by address
Create result array	merge, union, intersection, difference	Use a third array/result array

This table is the main mental model for the final project.

Part A: Core Project Setup

13. Step 1: Define the Structure

Start the project by defining the array structure.

```
#define MAX 50

struct Array {
    int A[MAX];
    int size;
    int length;
};
```

For testing, you can create an array like this:

```
struct Array arr = {{2, 3, 4, 5, 6}, MAX, 5};
```

This gives:

```
A = [2, 3, 4, 5, 6]
size = 50
length = 5
```

14. Step 2: Add the Display Function

Display prints all valid elements.

```
void display(struct Array arr) {
    for (int i = 0; i < arr.length; i++) {
        printf("%d ", arr.A[i]);
    }
    printf("\n");
}
```

Important points:

- It uses `length`, not `size`.
- It only reads the array.
- It can receive the structure by value.

Time complexity:

$O(N)$

Because it visits every valid element once.

15. Step 3: Add Append

Append adds a value at the end of the logical array.

The correct position is:

`arr->A[arr->length]`

Code:

```
void append(struct Array *arr, int x) {
    if (arr->length < arr->size) {
        arr->A[arr->length] = x;
        arr->length++;
    }
}
```

Main rule:

Append always inserts at index `length`.

Example:

```
Before: [2, 3, 4]
length = 3
append 10
After: [2, 3, 4, 10]
length = 4
```

Time complexity:

$O(1)$

Because append does not shift elements.

16. Step 4: Add Insert

Insert places a value at a specific index.

If the index already contains a value, elements must shift right.

Code:

```
void insert(struct Array *arr, int index, int x) {
    if (index >= 0 && index <= arr->length && arr->length < arr->size) {
        for (int i = arr->length; i > index; i--) {
            arr->A[i] = arr->A[i - 1];
        }
        arr->A[index] = x;
        arr->length++;
    }
}
```

Valid index rule for insert:

$0 \leq \text{index} \leq \text{length}$

Why is `index == length` allowed?

Because inserting at `length` means inserting at the end.

That is the same as append.

Time complexity:

Case	Situation	Time
Best	Insert at end	$O(1)$
Worst	Insert at beginning	$O(N)$

17. Insert Dry Run

Starting array:

```
A = [2, 3, 4, 5, 6]
length = 5
```

Call:

```
insert(&arr, 2, 10);
```

Goal:

```
Insert 10 at index 2
```

Trace:

Step	Action	Array State
Start	Original array	[2, 3, 4, 5, 6, _]
1	Move A[4] to A[5]	[2, 3, 4, 5, 6, 6]
2	Move A[3] to A[4]	[2, 3, 4, 5, 5, 6]
3	Move A[2] to A[3]	[2, 3, 4, 4, 5, 6]
4	Put 10 at index 2	[2, 3, 10, 4, 5, 6]
5	Increase length	length = 6

Final array:

[2, 3, 10, 4, 5, 6]

18. Step 5: Add Delete

Delete removes the value at a given index.

After deleting, elements must shift left to close the gap.

Code:

```
int Delete(struct Array *arr, int index) {
    int x = 0;

    if (index >= 0 && index < arr->length) {
        x = arr->A[index];

        for (int i = index; i < arr->length - 1; i++) {
            arr->A[i] = arr->A[i + 1];
        }

        arr->length--;
    }

    return x;
}
```

Valid index rule for delete:

$0 \leq \text{index} < \text{length}$

Why is `index == length` not allowed?

Because `index == length` is the first free slot, not an existing value.

Time complexity:

Case	Situation	Time
Best	Delete last element	$O(1)$
Worst	Delete first element	$O(N)$

19. Delete Dry Run

Starting array:

```
A = [2, 3, 4, 5, 6]
length = 5
```

Call:

```
Delete(&arr, 2);
```

Target value:

```
A[2] = 4
```

Trace:

Step	Action	Array State
1	Save deleted value <code>x = 4</code>	<code>[2, 3, 4, 5, 6]</code>
2	Move <code>A[3]</code> to <code>A[2]</code>	<code>[2, 3, 5, 5, 6]</code>
3	Move <code>A[4]</code> to <code>A[3]</code>	<code>[2, 3, 5, 6, 6]</code>
4	Decrease length	<code>length = 4</code>

Logical final array:

```
[2, 3, 5, 6]
```

The extra duplicate `6` at the end does not matter because it is outside `length`.

Part B: Searching Functions

20. Linear Search

Linear search checks elements one by one from left to right.

Code:

```
int linearSearch(struct Array arr, int key) {
    for (int i = 0; i < arr.length; i++) {
        if (arr.A[i] == key) {
            return i;
        }
    }
    return -1;
}
```

Important points:

- `key` is the value being searched.
- Return the index if found.
- Return `-1` if not found.
- Search only from `0` to `length - 1`.

Time complexity:

Case	Time
Best, key at index 0	$O(1)$
Worst, key at last index	$O(N)$
Unsuccessful search	$O(N)$
Average case	$O(N)$

21. Transposition Search

Transposition improves repeated linear searches.

When the key is found, it swaps the key with the previous element.

Code:

```
int transpositionSearch(struct Array *arr, int key) {
    for (int i = 0; i < arr->length; i++) {
        if (arr->A[i] == key) {
            if (i > 0) {
                swap(&arr->A[i], &arr->A[i - 1]);
                return i - 1;
            }
        }
    }
}
```



```

        return i;
    }
}
return -1;
}

```

Example:

```

Before: [2, 3, 4, 5, 6]
Search key: 5
After: [2, 3, 5, 4, 6]

```

The key moves one step closer to the front.

Important:

Transposition changes the array order, so it must use pass by address.

22. Move-to-Front Search

Move-to-front also improves repeated linear searches.

When the key is found, it swaps the key directly with the first element.

Code:

```

int moveToFrontSearch(struct Array *arr, int key) {
    for (int i = 0; i < arr->length; i++) {
        if (arr->A[i] == key) {
            swap(&arr->A[i], &arr->A[0]);
            return 0;
        }
    }
    return -1;
}

```

Example:

```

Before: [2, 3, 4, 5, 6]
Search key: 5
After: [5, 3, 4, 2, 6]

```

The key moves directly to the front.

Important warning:

Do not use transposition or move-to-front on an array that must stay sorted.

They can break sorted order.

23. Binary Search

Binary search is faster than linear search, but it has one strict rule:

Binary search works only on sorted arrays.

Code:

```
int BinarySearch(struct Array arr, int key) {
    int low = 0;
    int high = arr.length - 1;
    int mid;

    while (low <= high) {
        mid = (low + high) / 2;

        if (key == arr.A[mid]) {
            return mid;
        } else if (key < arr.A[mid]) {
            high = mid - 1;
        } else {
            low = mid + 1;
        }
    }

    return -1;
}
```

Pointer meanings:

Pointer	Meaning
low	Start of current search area
high	End of current search area

Pointer	Meaning
<code>mid</code>	Middle index

Time complexity:

Case	Time
Best	$O(1)$
Worst	$O(\log N)$
Unsuccessful	$O(\log N)$
Average	$O(\log N)$

24. Recursive Binary Search

Recursive binary search uses the same logic as iterative binary search, but it calls itself with a smaller search area.

Code:

```
int RBinSearch(int A[], int low, int high, int key) {
    if (low <= high) {
        int mid = (low + high) / 2;

        if (key == A[mid]) {
            return mid;
        } else if (key < A[mid]) {
            return RBinSearch(A, low, mid - 1, key);
        } else {
            return RBinSearch(A, mid + 1, high, key);
        }
    }

    return -1;
}
```

Base case:

```
low > high
```

This means the search area is empty.

Time complexity:

$O(\log N)$

Space complexity:

$O(\log N)$

Why?

Because recursive calls use stack memory.

The iterative version is usually preferred because it uses only $O(1)$ extra space.

Part C: Basic Read, Update, and Aggregate Functions

25. Get

Get returns the value at a valid index.

Code:

```
int get(struct Array arr, int index) {
    if (index >= 0 && index < arr.length) {
        return arr.A[index];
    }
    return -1;
}
```

Valid index rule:

$0 \leq \text{index} < \text{length}$

Time complexity:

$O(1)$

Because array index access is direct.

26. Set

Set changes the value at a valid index.

Code:

```
void set(struct Array *arr, int index, int x) {  
    if (index >= 0 && index < arr->length) {  
        arr->A[index] = x;  
    }  
}
```

Example:

```
Before: [2, 4, 6, 8]  
set index 1 to 20  
After: [2, 20, 6, 8]
```

Time complexity:

```
O(1)
```

27. Max

Max finds the largest value.

Code:

```
int max(struct Array arr) {  
    int max = arr.A[0];  
  
    for (int i = 1; i < arr.length; i++) {  
        if (arr.A[i] > max) {  
            max = arr.A[i];  
        }  
    }  
}
```

```
    return max;
}
```

Time complexity:

$O(N)$

Because it scans all valid values.

28. Min

Min finds the smallest value.

Code:

```
int min(struct Array arr) {
    int min = arr.A[0];

    for (int i = 1; i < arr.length; i++) {
        if (arr.A[i] < min) {
            min = arr.A[i];
        }
    }

    return min;
}
```

Time complexity:

$O(N)$

29. Sum

Sum adds all valid values.

Code:

```
int sum(struct Array arr) {
    int total = 0;
```

```
    for (int i = 0; i < arr.length; i++) {  
        total = total + arr.A[i];  
    }  
  
    return total;  
}
```

Time complexity:

$O(N)$

30. Recursive Sum

Recursive sum adds values using recursion.

Code:

```
int RSum(struct Array arr, int n) {  
    if (n < 0) {  
        return 0;  
    }  
  
    return RSum(arr, n - 1) + arr.A[n];  
}
```

Call example:

`RSum(arr, arr.length - 1);`

Time complexity:

$O(N)$

Space complexity:

$O(N)$

Because recursion uses stack memory.

31. Average

Average is:

sum / length

Code:

```
float average(struct Array arr) {  
    return (float)sum(arr) / arr.length;  
}
```

Why use `(float)`?

Because without it, C may do integer division.

Example:

21 / 4 = 5
(float)21 / 4 = 5.25

Time complexity:

$O(N)$

Because average calls sum.

Part D: Reverse, Shift, and Rotate

32. Reverse Using Auxiliary Array

This method creates another array, copies values in reverse order, then copies them back.

Simple idea:


```
A = [2, 3, 4, 5]
B = [5, 4, 3, 2]
A = [5, 4, 3, 2]
```

Time complexity:

$O(N)$

Space complexity:

$O(N)$

Because it uses an extra array.

33. Reverse Using Swaps

This method reverses the array in-place.

Code:

```
void Reverse2(struct Array *arr) {
    int i, j;

    for (i = 0, j = arr->length - 1; i < j; i++, j--) {
        swap(&arr->A[i], &arr->A[j]);
    }
}
```

Example:

```
Before: [2, 3, 4, 5, 6]
After:  [6, 5, 4, 3, 2]
```

Time complexity:

$O(N)$

Space complexity:

$O(1)$

This version is usually preferred because it does not create another full array.

34. Left Shift

Left shift moves every element one position left.

The first value is lost.

The last position becomes `0`.

Code:

```
void leftShift(struct Array *arr) {  
    for (int i = 0; i < arr->length - 1; i++) {  
        arr->A[i] = arr->A[i + 1];  
    }  
    arr->A[arr->length - 1] = 0;  
}
```

Example:

```
Before: [6, 3, 8, 5, 9]  
After:  [3, 8, 5, 9, 0]
```

35. Right Shift

Right shift moves every element one position right.

The last value is lost.

The first position becomes `0`.

Code:

```
void rightShift(struct Array *arr) {  
    for (int i = arr->length - 1; i > 0; i--) {  
        arr->A[i] = arr->A[i - 1];  
    }
```

```
    }  
    arr->A[0] = 0;  
}
```

Example:

Before: [6, 3, 8, 5, 9]
After: [0, 6, 3, 8, 5]

36. Left Rotate

Left rotate moves every element one position left, but the first value moves to the end.

Code:

```
void leftRotate(struct Array *arr) {  
    int first = arr->A[0];  
  
    for (int i = 0; i < arr->length - 1; i++) {  
        arr->A[i] = arr->A[i + 1];  
    }  
  
    arr->A[arr->length - 1] = first;  
}
```

Example:

Before: [6, 3, 8, 5, 9]
After: [3, 8, 5, 9, 6]

37. Right Rotate

Right rotate moves every element one position right, but the last value moves to the front.

Code:

```
void rightRotate(struct Array *arr) {  
    int last = arr->A[arr->length - 1];
```

```

    for (int i = arr->length - 1; i > 0; i--) {
        arr->A[i] = arr->A[i - 1];
    }

    arr->A[0] = last;
}

```

Example:

Before: [6, 3, 8, 5, 9]
 After: [9, 6, 3, 8, 5]

Part E: Sorted Array Operations

38. Sorted Insertion

Sorted insertion inserts a value into an already sorted array while keeping the array sorted.

Code:

```

void insertSort(struct Array *arr, int x) {
    if (arr->length == arr->size) {
        return;
    }

    int i = arr->length - 1;

    while (i >= 0 && arr->A[i] > x) {
        arr->A[i + 1] = arr->A[i];
        i--;
    }

    arr->A[i + 1] = x;
    arr->length++;
}

```

Example:

Before: [4, 8, 13, 16, 20]
Insert: 18
After: [4, 8, 13, 16, 18, 20]

Time complexity:

Case	Time
Best, insert at end	O(1)
Worst, insert at beginning	O(N)

39. Check if Array Is Sorted

`isSorted` checks whether the array is in ascending order.

Code:

```
int isSorted(struct Array arr) {  
    for (int i = 0; i < arr.length - 1; i++) {  
        if (arr.A[i] > arr.A[i + 1]) {  
            return 0;  
        }  
    }  
    return 1;  
}
```

Return values:

Return	Meaning
1	Array is sorted
0	Array is not sorted

Time complexity:

Case	Time
Best, first pair is wrong	O(1)
Worst, array is sorted or mistake is at end	O(N)

40. Rearranging Negative and Positive Values

Rearrange moves negative values to the left and non-negative values to the right.

Example:

Before: [2, -3, 25, -15, -7, 10]
After: [-7, -3, -15, 25, 2, 10]

Important:

Rearrangement is not sorting.

It only separates values into groups.

Code:

```
void rearrange(struct Array *arr) {  
    int i = 0;  
    int j = arr->length - 1;  
  
    while (i < j) {  
        while (i < j && arr->A[i] < 0) {  
            i++;  
        }  
  
        while (i < j && arr->A[j] >= 0) {  
            j--;  
        }  
  
        if (i < j) {  
            swap(&arr->A[i], &arr->A[j]);  
        }  
    }  
}
```

Time complexity:

$O(N)$

Space complexity:

0(1)

Part F: Merge and Set Operations

41. Merge Two Sorted Arrays

Merge combines two sorted arrays into one sorted array.

Example:

```
A = [2, 6, 10, 15, 25]
B = [3, 4, 7, 18, 20]
Merged = [2, 3, 4, 6, 7, 10, 15, 18, 20, 25]
```

Important rule:

Both input arrays must be sorted.

Merge uses three pointers:

Pointer	Meaning
i	Current index in first array
j	Current index in second array
k	Current index in result array

42. Merge Code

```
struct Array* merge(struct Array *arr1, struct Array *arr2) {
    int i = 0, j = 0, k = 0;
    struct Array *arr3 = (struct Array *)malloc(sizeof(struct Array));

    while (i < arr1->length && j < arr2->length) {
        if (arr1->A[i] < arr2->A[j]) {
            arr3->A[k++] = arr1->A[i++];
        } else {
            arr3->A[k++] = arr2->A[j++];
        }
    }
}
```

```

    }
}

while (i < arr1->length) {
    arr3->A[k++] = arr1->A[i++];
}

while (j < arr2->length) {
    arr3->A[k++] = arr2->A[j++];
}

arr3->length = k;
arr3->size = MAX;

return arr3;
}

```

Important:

```
arr3->length = k;
```

Why?

Because `k` tells how many values were copied into the result array.

Time complexity:

```
O(m + n)
```

Space complexity:

```
O(m + n)
```

43. Why Merge Uses `malloc`

Inside `merge`, the result array is created like this:

```
struct Array *arr3 = (struct Array *)malloc(sizeof(struct Array));
```

This creates the result array in heap memory.

Why not do this?

```
struct Array arr3;  
return &arr3;
```

Because `arr3` would be a local variable.

After the function ends, local variables are destroyed.

Returning the address of a destroyed local variable is unsafe.

Simple memory:

```
Do not return the address of a local stack variable.  
Use heap memory or pass the result array from outside.
```

44. Union

Union returns all unique values from both arrays.

Example:

```
A = [2, 6, 10]  
B = [3, 6, 7]  
Union = [2, 3, 6, 7, 10]
```

The value `6` appears in both arrays, but it is copied only once.

Simple idea:

```
Union = everything, but common values appear once.
```

45. Union Code

```
struct Array* Union(struct Array *arr1, struct Array *arr2) {  
    int i = 0, j = 0, k = 0;  
    struct Array *arr3 = (struct Array *)malloc(sizeof(struct Array));
```

```

while (i < arr1->length && j < arr2->length) {
    if (arr1->A[i] < arr2->A[j]) {
        arr3->A[k++] = arr1->A[i++];
    } else if (arr2->A[j] < arr1->A[i]) {
        arr3->A[k++] = arr2->A[j++];
    } else {
        arr3->A[k++] = arr1->A[i++];
        j++;
    }
}

while (i < arr1->length) {
    arr3->A[k++] = arr1->A[i++];
}

while (j < arr2->length) {
    arr3->A[k++] = arr2->A[j++];
}

arr3->length = k;
arr3->size = MAX;

return arr3;
}

```

Time complexity for sorted arrays:

$O(m + n)$

46. Intersection

Intersection returns only values that are present in both arrays.

Example:

```

A = [2, 6, 10]
B = [3, 6, 7]
Intersection = [6]

```

Simple idea:

Intersection = common values only.

47. Intersection Code

```
struct Array* Intersection(struct Array *arr1, struct Array *arr2) {
    int i = 0, j = 0, k = 0;
    struct Array *arr3 = (struct Array *)malloc(sizeof(struct Array));

    while (i < arr1->length && j < arr2->length) {
        if (arr1->A[i] < arr2->A[j]) {
            i++;
        } else if (arr2->A[j] < arr1->A[i]) {
            j++;
        } else {
            arr3->A[k++] = arr1->A[i++];
            j++;
        }
    }

    arr3->length = k;
    arr3->size = MAX;

    return arr3;
}
```

Important:

Intersection does not copy leftovers.

Why?

If one array is finished, remaining values in the other array cannot be common.

Time complexity:

$O(m + n)$

48. Difference

Difference returns values in the first array that are not in the second array.

It is written as:

A - B

Example:

A = [2, 6, 10]
B = [3, 6, 7]
A - B = [2, 10]

Important:

Difference is directional.

A - B is not always the same as B - A.

Example:

A - B = [2, 10]
B - A = [3, 7]

49. Difference Code

```
struct Array* Difference(struct Array *arr1, struct Array *arr2) {
    int i = 0, j = 0, k = 0;
    struct Array *arr3 = (struct Array *)malloc(sizeof(struct Array));

    while (i < arr1->length && j < arr2->length) {
        if (arr1->A[i] < arr2->A[j]) {
            arr3->A[k++] = arr1->A[i++];
        } else if (arr2->A[j] < arr1->A[i]) {
            j++;
        } else {
            i++;
            j++;
        }
    }

    while (i < arr1->length) {
        arr3->A[k++] = arr1->A[i++];
    }
}
```

```
    }  
  
    arr3->length = k;  
    arr3->size = MAX;  
  
    return arr3;  
}
```

Important:

Difference copies leftovers only from the first array.

Why?

Because `A - B` keeps only values from `A`.

Time complexity:

$O(m + n)$

50. Membership

Membership checks whether a value exists in an array or set.

Question:

Is key present in the array?

If the array is unsorted, use linear search:

$O(N)$

If the array is sorted, use binary search:

$O(\log N)$

Simple memory:

```
Unsorted membership = linear search.  
Sorted membership = binary search.
```

Part G: Final Project Build Order

51. Recommended Build Order

Do not write everything at once.

Build the project step by step.

Step 1: Structure and Display

Start with:

```
struct Array  
display()
```

Make sure display prints only valid elements.

Step 2: Basic Update Operations

Add:

```
append()  
insert()  
Delete()
```

Test these carefully because many later operations depend on correct `length` handling.

Step 3: Searching Operations

Add:

```
linearSearch()  
transpositionSearch()  
moveToFrontSearch()
```

```
BinarySearch()  
RBinSearch()
```

Important:

Check sorted order before using binary search.

Step 4: Read and Aggregate Operations

Add:

```
get()  
set()  
max()  
min()  
sum()  
RSum()  
average()
```

Test valid and invalid indexes for `get` and `set`.

Step 5: Rearrangement Operations

Add:

```
Reverse1()  
Reverse2()  
leftShift()  
rightShift()  
leftRotate()  
rightRotate()
```

Use `Reverse2` as the preferred reverse method because it uses less extra space.

Step 6: Sorted and Set Operations

Add:

```
insertSort()  
isSorted()
```

```
rearrange()  
merge()  
Union()  
Intersection()  
Difference()  
membership()
```

These are more advanced because they rely on order, pointer movement, or result arrays.

52. Menu Design Strategy

A large menu can become messy.

So organize it clearly.

Example menu:

```
--- Array ADT Menu ---  
1. Display  
2. Append  
3. Insert  
4. Delete  
5. Linear Search  
6. Transposition Search  
7. Move-to-Front Search  
8. Binary Search  
9. Get  
10. Set  
11. Max  
12. Min  
13. Sum  
14. Recursive Sum  
15. Average  
16. Reverse using Swaps  
17. Left Shift  
18. Right Shift  
19. Left Rotate  
20. Right Rotate  
21. Sorted Insert  
22. Check Sorted  
23. Rearrange Negative and Positive  
24. Merge Two Sorted Arrays  
25. Union  
26. Intersection
```


27. Difference
0. Exit

You do not need to memorize this exact numbering.

The important idea is that every menu option should call one clear function.

Part H: Required Dry Runs

53. Why Dry Runs Matter

A dry run means tracing the code manually step by step.

Dry runs prove that you understand what the function is doing.

For the final project, you should be able to dry-run these operations:

1. Insert at index 0
2. Delete at index 0
3. Linear search success
4. Linear search failure
5. Binary search success
6. Binary search failure
7. Reverse using swaps
8. Sorted insertion
9. Rearranging negative and positive values
10. Merge two sorted arrays
11. Union
12. Intersection
13. Difference

54. Dry Run: Insert at Index 0

Starting array:

[2, 3, 4, 5]

Insert 10 at index 0.

All elements shift right:

```
A[4] = A[3]
A[3] = A[2]
A[2] = A[1]
A[1] = A[0]
A[0] = 10
```

Final array:

```
[10, 2, 3, 4, 5]
```

This is worst case insert.

Time:

```
O(N)
```

55. Dry Run: Delete at Index 0

Starting array:

```
[2, 3, 4, 5]
```

Delete index .

All elements shift left:

```
A[0] = A[1]
A[1] = A[2]
A[2] = A[3]
length--
```

Final logical array:

```
[3, 4, 5]
```

This is worst case delete.

Time:

$O(N)$

56. Dry Run: Binary Search Failure

Array:

[2, 4, 6, 8, 10, 12, 14]

Search key:

5

Trace:

Step	low	high	mid	A[mid]	Decision
1	0	6	3	8	5 < 8, search left
2	0	2	1	4	5 > 4, search right
3	2	2	2	6	5 < 6, search left
Stop	2	1	-	-	low > high

Return:

-1

Meaning:

The key is not present.

57. Dry Run: Merge

Arrays:

A = [2, 6, 10]
B = [3, 4, 7]

Trace:

Step	Compare	Copy	Result
1	2 vs 3	2	[2]
2	6 vs 3	3	[2, 3]
3	6 vs 4	4	[2, 3, 4]
4	6 vs 7	6	[2, 3, 4, 6]
5	10 vs 7	7	[2, 3, 4, 6, 7]
6	B finished	10	[2, 3, 4, 6, 7, 10]

Final result:

[2, 3, 4, 6, 7, 10]

Part I: Complexity Summary

58. Final Project Complexity Table

Operation	Time Complexity	Space Complexity	Notes
Display	$O(N)$	$O(1)$	Visits valid elements
Append	$O(1)$	$O(1)$	Directly adds at <code>length</code>
Insert	$O(N)$ worst	$O(1)$	May shift elements right
Delete	$O(N)$ worst	$O(1)$	May shift elements left
Linear Search	$O(N)$	$O(1)$	Checks one by one
Transposition Search	$O(N)$	$O(1)$	May swap with previous value
Move-to-Front Search	$O(N)$	$O(1)$	May swap with first value
Binary Search	$O(\log N)$	$O(1)$	Requires sorted array
Recursive Binary Search	$O(\log N)$	$O(\log N)$	Uses call stack
Get	$O(1)$	$O(1)$	Direct index access

Operation	Time Complexity	Space Complexity	Notes
Set	$O(1)$	$O(1)$	Direct index update
Max	$O(N)$	$O(1)$	Scans all values
Min	$O(N)$	$O(1)$	Scans all values
Sum	$O(N)$	$O(1)$	Adds all values
Recursive Sum	$O(N)$	$O(N)$	Uses call stack
Average	$O(N)$	$O(1)$	Calls sum
Reverse with auxiliary array	$O(N)$	$O(N)$	Uses extra array
Reverse with swaps	$O(N)$	$O(1)$	In-place
Shift	$O(N)$	$O(1)$	Moves elements
Rotate	$O(N)$	$O(1)$	Moves elements and saves one value
Sorted Insert	$O(N)$ worst	$O(1)$	Shifts larger values right
isSorted	$O(N)$ worst	$O(1)$	Checks neighboring pairs
Rearrange	$O(N)$	$O(1)$	Uses two pointers
Merge	$O(m + n)$	$O(m + n)$	Uses result array
Union	$O(m + n)$ sorted	$O(m + n)$	Uses merge-style logic
Intersection	$O(m + n)$ sorted	$O(m + n)$ or less	Copies common values
Difference	$O(m + n)$ sorted	$O(m)$ or more depending on allocation	Copies values from first array only

Part J: Common Beginner Mistakes

59. Mistake 1: Confusing `size` and `length`

Wrong:

```
for (int i = 0; i < arr.size; i++)
```

Correct:

```
for (int i = 0; i < arr.length; i++)
```

Use `length` when working with real data.

60. Mistake 2: Forgetting Capacity Checks

Before append or insert, check whether the array has space.

Correct:

```
if (arr->length < arr->size)
```

Without this, the program may write outside the array.

61. Mistake 3: Using Binary Search on Unsorted Arrays

Binary search only works on sorted arrays.

Wrong:

```
Use binary search on [5, 2, 8, 1]
```

Correct:

```
Use binary search only on [1, 2, 5, 8]
```

If the array may not be sorted, check first using `isSorted()`.

62. Mistake 4: Returning Address of a Local Variable

Wrong:

```
struct Array arr3;  
return &arr3;
```

This is unsafe because `arr3` disappears when the function ends.

Correct:

```
struct Array *arr3 = malloc(sizeof(struct Array));  
return arr3;
```

Or pass the result array from outside.

63. Mistake 5: Forgetting to Set Result Length

For merge and set operations, always set:

```
arr3->length = k;
```

Why?

Because `k` tells how many values were actually copied.

Without this, display may print wrong or unused values.

64. Mistake 6: Wrong Pointer Notation

If the function receives a normal structure:

```
struct Array arr
```

Use dot:

```
arr.length  
arr.A[i]
```

If the function receives a pointer:

```
struct Array *arr
```

Use arrow:

```
arr->length  
arr->A[i]
```

65. Mistake 7: Shifting in the Wrong Direction

For insertion, shift from right to left.

Correct:

```
for (int i = arr->length; i > index; i--)
```

For deletion, shift from left to right.

Correct:

```
for (int i = index; i < arr->length - 1; i++)
```

Simple memory:

Insert creates space by shifting right from the end.
Delete closes space by shifting left from the deleted index.

Part K: Final Assessment Checklist

66. Code Checklist

Your final project should include:

- struct Array
- display
- append
- insert
- Delete
- linearSearch
- transpositionSearch
- moveToFrontSearch
- BinarySearch

- RBinSearch
- get
- set
- max
- min
- sum
- RSum
- average
- Reverse1
- Reverse2
- leftShift
- rightShift
- leftRotate
- rightRotate
- insertSort
- isSorted
- rearrange
- merge
- Union
- Intersection
- Difference
- membership check
- menu system

67. Logic Checklist

Your program should correctly:

- use `length` for valid elements
 - use `size` for capacity
 - avoid printing unused slots
 - validate indexes before access
 - prevent insertion when the array is full
 - return `-1` for failed searches
 - return `0` or another clear value for invalid delete if using the given beginner style
 - use binary search only on sorted arrays
 - use result arrays for merge and set operations
 - set result array length correctly
 - free heap memory when needed
-

68. Dry Run Checklist

You should be able to manually trace:

- append
 - insert at beginning
 - insert in middle
 - delete at beginning
 - delete in middle
 - linear search success
 - linear search failure
 - binary search success
 - binary search failure
 - reverse using swaps
 - left shift
 - right rotate
 - sorted insertion
 - rearrange negative and positive values
 - merge
 - union
 - intersection
 - difference
-

69. Complexity Checklist

You should be able to explain why:

- append is $O(1)$
 - insert can be $O(N)$
 - delete can be $O(N)$
 - linear search is $O(N)$
 - binary search is $O(\log N)$
 - get and set are $O(1)$
 - max, min, sum, and average are $O(N)$
 - reverse with swaps is $O(N)$ time and $O(1)$ space
 - merge is $O(m + n)$
 - sorted union, intersection, and difference are $O(m + n)$
-

70. Final Project Summary

The final phase is the point where all previous phases become one complete project.

You are not just writing separate functions anymore.

You are building a small system.

That system has:

Data representation:

A, size, length

Operations:

append, insert, delete, search, reverse, merge, set operations, and more

Controller:

menu-driven program

Safety rules:

index validation, capacity checks, sorted preconditions

Proof of understanding:

dry runs and complexity analysis

Final simple memory:

The Array ADT project is complete when the user can choose operations from a menu,
the correct function runs safely,
the array stays logically valid,
and you can explain what happens step by step.

71. Beginner-Friendly Final Picture

Think of the Array ADT like a small machine.

The array structure is the machine body:

```
struct Array {  
    int A[MAX];  
    int size;  
    int length;  
};
```

The functions are the machine tools:

append, insert, delete, search, reverse, merge, union, intersection...

The menu is the control panel:

Press 1 for display
Press 2 for append
Press 3 for insert
...

The user chooses a button.

The correct tool runs.

The array changes or produces a result.

That is the complete final phase.